

EC9540 - HUMAN COMPUTER INTERACTION

Lab 05

DE COSTA M.S.M.

2022/E/092

SENARATHNA S.A.D.H.D.

2022/E/141

MediLink: A Community Healthcare Information and Support Platform

Introduction

Healthcare services are essential for improving people's quality of life. However, many patients still face difficulties when accessing hospital services due to manual appointment booking, limited information about doctor and medicine availability and time-consuming payment processes. These challenges often lead to unnecessary hospital visits, long waiting times and inconvenience for patients.

To address these issues, we propose **MediLink**, a web-based healthcare platform designed to improve the patient experience through a simple and user-friendly interface. Based on the findings of our user requirements survey, the scope of the system focuses on three core patient services:

- Checking doctor availability and booking appointments.
- Checking medicine availability before visiting the hospital.
- Making secure online payments while providing patient details.

These features were selected because they directly address the main problems identified through the questionnaire survey. They also provide a complete and convenient user journey, allowing patients to check doctor availability, book appointments, verify medicine availability and complete payments online without unnecessary hospital visits. The system is designed following Human-Computer Interaction (HCI) principles to ensure usability, efficiency and a positive user experience.

Back-end implementation overview

Technology stack and rationale

MediLink is implemented on the MERN stack. The table below records not only what was selected but why, since several of these choices were driven directly by requirements arising from the interface design.

Layer	Technology	Role in MediLink and reason for selection
Runtime	Node.js 18+	Single-language development across client and server. Its non-blocking I/O model suits an API whose workload is dominated by database round trips rather than computation.
Web framework	Express 4.21	Minimal, unopinionated routing and middleware. The middleware model allows cross-cutting concerns such as authentication, validation and rate limiting to be composed per route rather than repeated inside handlers.
Database	MongoDB 6+	A document model suits the domain: an appointment naturally carries nested patient details, a doctor's snapshot and a reschedule history as one atomic record, avoiding joins across five tables to render one screen.

ODM	Mongoose 8.9	Provides schema definition, type casting, validation, virtual, middleware hooks and index declaration over MongoDB's otherwise schemeless collections. It is what gives the data tier its integrity guarantees.
Authentication	jsonwebtoken 9	Stateless bearer tokens. No server-side session store is required, which keeps horizontal scaling straightforward.
Password hashing	bcryptjs 2.4	Adaptive hashing with a configurable cost factor (12 rounds). Deliberately slow, which is the property that makes offline brute-force attack impractical.
Validation	Express-validator 7	Declarative, per-route validation chains that run before any controller execute, producing field-level error messages the react forms display directly.
Security	Helmet, cors, Express-rate-limit, Express-mongo-sanitize, hpp	Security response headers, a cross-origin whitelist, brute-force throttling, NoSQL operator-injection stripping and http parameter-pollution protection.
Date handling	Dayjs 1.11	Strict-format parsing of appointment dates and times, used throughout the availability engine.
Logging	Morgan + custom logger	Http access logging with structured, level-filtered application logs that emit json in production.

Three-tier architecture

The system follows a conventional three-tier separation. The presentation tier is the React application, which holds no business rules and never contacts MongoDB directly. The application tier is the Express API, which owns all validation, authorization and business logic. The data tier is MongoDB accessed exclusively through Mongoose models. The only channel between the first two tiers is HTTP carrying JSON, which means the React client could be replaced by a mobile application without any server-side change.

Layered server structure

Within the application tier the server is further divided so that each module has a single responsibility. A request passes through the layers in a fixed order and each layer is permitted to call only the layer beneath it.

```
server/
├── src/
│   ├── config/    env.js · database.js · logger.js
│   ├── models/    User · Hospital · Doctor · Medicine · Appointment · Payment
│   ├── validators/ auth · appointment · payment · query validation chains
│   ├── controllers/ auth · doctor · medicine · appointment · payment · dashboard
│   ├── services/   availability · appointment · payment (business rules)
│   ├── routes/     versioned route definitions mounted at /api/v1
│   ├── middleware/ auth · validate · error · requestId · rateLimit
│   ├── utils/      ApiError · apiResponse · asyncHandler · token · datetime
│   ├── docs/       openapi.js (OpenAPI 3.0 specification)
│   ├── app.js      Express application assembly
│   └── server.js    process lifecycle: connect, listen, graceful shutdown
├── seed/seed.js    reference data and demonstration accounts
├── tests/api.test.js automated unit, contract and integration tests
└── .env.example     documented configuration template
```

Two aspects of this structure are worth drawing out.

Controllers are deliberately thin

A controller reads the already-validated request, calls a service or a model and hands the result to the shared response helper. It contains no business rules. The entire booking controller is under ninety lines because the rules governing what constitute a valid booking live in the appointment service.

Services own the business rules

Three service modules carry the domain logic. The availability service generates bookable slots. The appointment service enforces the booking lifecycle, reservation, confirmation, rescheduling, cancellation and the notice period attached to each. The payment service encapsulates the gateway interaction. Because these rules sit behind a plain function interface rather than inside an HTTP handler, they can be tested directly and would remain correct if the same operations were later triggered by a scheduled job or an administrative tool.

Application assembly is separate from process lifecycle

app.js builds and exports the configured Express application without opening a port; server.js connects to MongoDB, starts the HTTP listener and installs signal handlers for graceful shutdown. This separation is what allows the automated test suite to import and exercise the real application over HTTP without requiring a database or a fixed port.

Middleware pipeline

Every request traverses the same ordered pipeline before reaching a route handler. The order is not arbitrary; it is the security spine of the service. Sanitization must run after the body has been parsed but before any handler reads it and the error handler must be registered last so that it catches failures raised anywhere above it.

	Middleware	Purpose
1	requestId	Assigns a UUID correlation identifier, returned in the X-Request-Id header and echoed in every error response so a user-reported fault can be traced to a specific log entry.
2	morgan	HTTP access logging routed through the application logger.
3	helmet	Sets security response headers including X-Content-Type-Options, X-Frame-Options and Referrer-Policy.
4	cors	Restricts browser access to the origins listed in CORS_ORIGIN. Any other origin is rejected.
5	express.json	Parses the JSON body with a 100 KB ceiling, limiting the surface for payload-based denial of service.
6	express-mongo-sanitize	Strips keys containing \$ or . so a crafted body cannot inject a MongoDB query operator for example, submitting { "email": { "\$ne": null } } to bypass a login check.
7	hpp	Prevents HTTP parameter pollution such as ?specialty=a&specialty=b arriving as an array where a string is expected.
8	compression	gzip response compression, which materially reduces payload size on the doctor and medicine list endpoints.
9	rateLimit	Tiered throttling: 300 requests per 15 minutes for general API use, 20 per 15 minutes on authentication endpoints, and 12 per minute on booking and payment endpoints.
10	routes	Versioned resource routers mounted under /api/v1.
11	notFoundHandler	Converts an unmatched route into a 404 expressed in the standard error envelope.
12	errorHandler	Normalises every failure, thrown ApiError, Mongoose validation error, duplicate key, cast error or unexpected exception into one consistent JSON shape.

REST API surface

All resources are mounted under a version prefix so that a future breaking change can be introduced as /api/v2 without disrupting clients already in use. Twenty-one endpoints are implemented across seven resource groups.

Method	Endpoint	Auth	Description
GET	/api/v1/health	-	Service and database health probe; returns 503 when MongoDB is unreachable.
POST	/api/v1/auth/register	-	Create a patient account and issue an access token.
POST	/api/v1/auth/login	-	Authenticate and issue an access token.
GET	/api/v1/auth/me	JWT	Current account profile.
PATCH	/api/v1/auth/me	JWT	Update name and phone number.

POST	/api/v1/auth/change-password	JWT	Change password, re-authenticating with the current one.
GET	/api/v1/doctors	-	Search doctors by name, specialty and hospital; paginated, each result carrying its next available slot.
GET	/api/v1/doctors/specialties	-	Distinct specialties with doctor counts, used to populate the filter dropdown.
GET	/api/v1/doctors/:id	-	Doctor profile, consultation fee and seven-day availability calendar.
GET	/api/v1/doctors/:id/availability	-	Generated appointment slots for a given date.
GET	/api/v1/doctors/:id/calendar	-	Free-slot count per day across a date range.
GET	/api/v1/appointments	JWT	Patient's appointments, split into upcoming and previous.
POST	/api/v1/appointments	JWT	Reserve a slot; create the booking in pending_payment state.
GET	/api/v1/appointments/:id	JWT	Single appointment, scoped to the owning patient.
PATCH	/api/v1/appointments/:id/reschedule	JWT	Move the appointment to a different slot.
PATCH	/api/v1/appointments/:id/cancel	JWT	Cancel and release the slot; refunds a paid fee.
POST	/api/v1/payments	JWT	Authorise the consultation fee and confirm the appointment.
GET	/api/v1/payments	JWT	Payment history for the signed-in patient.
GET	/api/v1/payments/:id/receipt	JWT	Formatted receipt for a payment.
GET	/api/v1/medicines	-	Search hospital pharmacy stock with derived availability status.
GET	/api/v1/medicines/summary	-	Stock position aggregated per hospital.
GET	/api/v1/hospitals	-	Active hospitals, used to populate hospital filters.
GET	/api/v1/dashboard	JWT	Aggregated dashboard payload in a single request.

Response and error contract

Consistency between endpoints was treated as a design requirement rather than a convention. Every successful response uses the same envelope, which means the React service layer unwraps results with a single interceptor instead of special casing each call.

```
{
  "success": true,
  "message": "2 doctors matched your search.",
  "data": [ ... ],
  "meta": { "page": 1, "limit": 12, "total": 2, "totalPages": 1 }
}
```

Failures use a parallel shape. Validation failures additionally carry field-level detail, which the React forms map directly onto the relevant input so the patient sees the message beside the field that caused it.

```
{
  "success": false,
  "message": "Some of the submitted information is not valid.",
  "requestId": "93769fb1-23e0-4bf8-997c-64d8c569f480",
  "errors": [
    { "field": "email", "message": "Please enter a valid email address." },
    { "field": "password", "message": "Must contain at least 8 characters." }
  ]
}
```

HTTP status codes are used according to their defined semantics rather than defaulting to 200 or 500:

Status	Meaning in MediLink
200 / 201	Request succeeded; 201 when a resource was created (an appointment reservation or a payment).
400	Malformed request unparseable JSON, or an invalid identifier format.
401	Missing, forged or expired access token.
403	Authenticated, but the account role does not permit the action, or the account is deactivated.
404	Resource not found. Also returned when a patient requests another patient's appointment, since confirming its existence would itself leak information.
409	Conflict with current state — the slot was taken moments ago, the appointment is already paid, or a cancellation falls inside the notice period.
422	The request was well-formed but failed field validation.
402	Payment declined by the issuer. A business outcome rather than a fault, so the slot remains reserved and the patient may retry.
429	Rate limit exceeded.
503	The API is running but cannot reach MongoDB.

Database and data management

The database holds six collections. Four are reference data describing the healthcare network users, hospitals, doctors and medicine stock and two are transactional: appointments and payments.

The Lab 04 prototype held doctors and medicines flat arrays with a free-text hospital name repeated on each record, and it had no representation of a payment at all. The schema below normalizes hospitals into their own collection, replaces the free-text medicine status with a quantity-derived value, and introduces payments as first-class records so that a consultation fee has an auditable history rather than being a Boolean flag on the booking.

Entity relationship model

Relationships are modelled by reference rather than by embedding wherever a document is referenced from more than one place. A hospital is referenced by doctors, medicine stock records and appointments, so embedding it would triple the storage of a hospital's address and, more importantly, make a change of telephone number require three separate updates.

One deliberate exception exists. Each appointment embeds a doctor's Snapshot containing the doctor's name, specialty and hospital name at the moment of booking, alongside the reference to the live doctor's record. This is not redundancy by oversight. A patient's historic appointment record should continue to show the details that applied when the appointment was made and the consultation fee is snapshotted for the same reason: a later change to a doctor's fee must not silently rewrite the amount a patient already agreed to pay.

Collection schemas

users

A single collection holds every actor in the system, with the role field acting as a discriminator that drives route authorization. The password hash carries select: false, which excludes it from query results by default so it cannot be leaked by a controller that forgets to project fields explicitly.

Field	Type	Constraints	Notes
_id	ObjectId	Primary key	Generated by MongoDB.
name	String	Required, 2-80 chars	Patient's full name.
email	String	Required, unique, lowercase	Login identifier; validated against an email pattern.
phone	String	Required, pattern-matched	Contact number, 9-15 digits with optional country prefix.
passwordHash	String	Required, select: false	bcrypt hash at cost factor 12. Never returned by any endpoint.
role	Enum	patient doctor staff admin	Default patient. Drives the authorize() middleware.
doctorProfile	ObjectId	→ doctors, nullable	Set only when the account belongs to a practising doctor.

isActive	Boolean	Default true	Soft deactivation; an inactive account is rejected at authentication.
lastLoginAt	Date	Nullable	Updated on each successful sign-in.
createdAt / updatedAt	Date	Automatic	Mongoose timestamps.

hospitals

Normalizing hospitals removes the duplicated free-text names present in the earlier prototype and gives doctors, stock records and appointments one authoritative record to reference.

Field	Type	Constraints	Notes
_id	ObjectId	Primary key	
name	String	Required, unique	Display name, e.g. MediLink General Hospital.
code	String	Required, unique, uppercase	Short code used in seeding and reporting, e.g. MLGH.
city	String	Required	Used for grouping and display.
address	String	Optional	Shown on the doctor profile screen.
contactNumber	String	Optional	Hospital switchboard.
isActive	Boolean	Default true	Inactive hospitals are excluded from filter dropdowns.

doctors

The weekly Schedule sub-document is the single most consequential design decision in the schema. Rather than storing a list of free appointment slots which would begin going stale the moment the first booking was made the collection stores the doctor's recurring consulting pattern. Concrete slots are generated on demand by the availability service, described in Section 3.4.

Field	Type	Constraints	Notes
_id	ObjectId	Primary key	
name	String	Required, indexed	Display name.
specialty	String	Required, indexed	Cardiology, Dermatology, Pediatrics and so on.
hospital	ObjectId	→ hospitals, required	Where the doctor consults.
qualifications	String	Optional	MBBS, MD and similar, shown on the profile screen.
bio	String	Max 600 chars	Professional summary.
consultationFee	Number	Required, ≥ 0	In LKR. Snapshotted onto each appointment at booking.
yearsOfExperience	Number	≥ 0	Shown on the doctor profile.

slotDurationMinutes	Number	10–120, default 30	Governs how schedule blocks are divided into slots.
weeklySchedule[]	Subdocument[]	dayOfWeek 0-6, startTime, endTime HH:mm	Recurring consulting blocks. Validated so each block starts before it ends.
blockedDates[]	String[]	YYYY-MM-DD	Individual dates on which the doctor is not consulting, e.g. leave.
isAcceptingAppointments	Boolean	Default true, indexed	Hides a doctor from booking without deleting the record.
rating	Number	0–5	Displayed on search result cards.

medicines

Stock is modelled as an integer quantity against a reorder level rather than as a free-text status string. The status shown to the patient is derived from those numbers through a Mongoose virtual, which means the badge on screen can never disagree with the underlying inventory figure, a class of inconsistency that was structurally possible in the earlier prototype, where the status was stored independently of any quantity.

Field	Type	Constraints	Notes
_id	ObjectId	Primary key	
name	String	Required, indexed	Brand or common name, e.g. Paracetamol.
genericName	String	Optional	Also searched, so a generic-name query finds the brand.
strength	String	Optional	e.g. 500mg.
form	Enum	tablet capsule syrup injection cream drops inhaler	Presentation.
hospital	ObjectId	→ hospitals, required	Which pharmacy holds this stock.
quantityAvailable	Number	Required, ≥ 0	Units on the shelf.
reorderLevel	Number	Required, ≥ 0	Threshold below which stock counts as limited.
unitPrice	Number	≥ 0	Indicative price in LKR.
requiresPrescription	Boolean	Default false	Surfaced as a warning on the results screen.
stockUpdatedAt	Date	Default now	Drives the "updated N minutes ago" line.

stockStatus	Virtual	Derived	in_stock low_stock out_of_stock, computed from quantity and reorder level.
lastUpdatedText	Virtual	Derived	Relative time string computed from stockUpdatedAt.

appointments

The central transactional collection. Three design points distinguish it from a naive booking record.

- Dual time representation. The appointment date and time are stored as strings (YYYY-MM-DD and HH:mm) for simple equality matching against the availability engine, and additionally as a single scheduledAt Date so that range queries such as "my upcoming appointments" and chronological sorting are index-supported.
- Immutable snapshots. doctorSnapshot and consultationFee freeze the details that applied at booking time.
- Audit trail. Every reschedule appends an entry to rescheduleHistory and cancellation records who cancelled, when and why, so the record explains its own history.

Field	Type	Constraints	Notes
_id	ObjectId	Primary key	
confirmationNumber	String	Required, unique	Patient-facing reference, e.g. MED-7K4Q2XPA.
patient	ObjectId	→ users, required	Owning account; every query is scoped by it.
doctor	ObjectId	→ doctors, required	Live doctor record.
hospital	ObjectId	→ hospitals, required	Denormalised from the doctor for direct filtering.
doctorSnapshot	Subdocument	name, specialty, hospitalName	Immutable copy taken at booking.
appointmentDate	String	YYYY-MM-DD	Matched against generated slots.
appointmentTime	String	HH:mm	24-hour slot start time.
scheduledAt	Date	Required, indexed	Combined date and time for range queries.
durationMinutes	Number	Default 30	Copied from the doctor's slot duration.
patientDetails	Subdocument	fullName, phone, email, notes	Captured on the Patient Details screen; may differ from the account holder.
consultationFee	Number	Required, ≥ 0	Snapshotted fee, in LKR.
status	Enum	pending_payment confirmed completed cancelled	Booking lifecycle state.

paymentStatus	Enum	pending paid failed refunded	Financial state, tracked separately from the booking state.
payment	ObjectId	→ payments, nullable	Set once a payment is approved.
cancellation	Subdocument	cancelledAt, cancelledBy, reason	Populated on cancellation.
rescheduleHistory[]	Subdocument[]	previous/new date and time	Append-only audit of every move.

payments

Payments are recorded as separate documents rather than as fields on the appointment, which gives each transaction its own reference, timestamp, gateway response and refund state, and allows a failed attempt to be retained for audit alongside the successful one that followed it.

The collection deliberately stores no sensitive cardholder data. Only the card brand and the last four digits are retained sufficient to identify the payment on a receipt while keeping MediLink outside PCI-DSS cardholder-data scope. The full card number, expiry date and security code are used to obtain an authorisation and are then discarded.

Field	Type	Constraints	Notes
_id	ObjectId	Primary key	
transactionReference	String	Required, unique	e.g. TXN-20260916-9D3FQ2.
appointment	ObjectId	→ appointments, required	The booking being paid for.
patient	ObjectId	→ users, required	Payer.
amount	Number	Required, ≥ 0	Equals the appointment's snapshotted consultation fee.
currency	String	Default LKR	
method	Enum	card online_banking	Selected on the payment screen.
status	Enum	pending paid failed refunded	Outcome of the authorisation.
instrument.cardBrand	String	Nullable	Visa, Mastercard, American Express.
instrument.last4	String	4 digits, nullable	Last four digits only. The full number is never persisted.
instrument.bankName	String	Nullable	Set for online banking payments.
gatewayResponse	Subdocument	code, message, processedAt	Acquirer response retained for reconciliation.
failureReason	String	Nullable	Decline reason shown to the patient.

refundedAt	Date	Nullable	Set when a paid appointment is cancelled.
------------	------	----------	---

Indexing strategy

Indexes were declared against the queries the interface actually issues rather than added speculatively. Each entry below states which screen it serves.

Collection	Index	Type	Query it supports
users	{ email: 1 }	Unique	Login lookup, and enforcement of one account per email address.
users	{ role: 1 }	Secondary	Role-scoped administrative queries.
doctors	{ specialty: 1, hospital: 1, isAcceptingAppointments: -1 }	Compound	The combined specialty and hospital filter on Find a Doctor.
doctors	{ name, specialty, qualifications }	Text	Free-text doctor name search.
medicines	{ name: 1, strength: 1, hospital: 1 }	Unique	Guarantees one stock record per medicine per hospital.
medicines	{ name, genericName }	Text	Medicine search by brand or generic name.
appointments	{ doctor: 1, appointmentDate: 1, appointmentTime: 1 }	Partial unique	Prevents double-booking. Applied only to statuses that still hold a slot — see Section 3.4.
appointments	{ patient: 1, scheduledAt: -1 }	Compound	My Appointments, the most frequent patient query.
appointments	{ confirmationNumber: 1 }	Unique	Lookup by the reference printed on the confirmation screen.
payments	{ patient: 1, createdAt: -1 }	Compound	Payment history, newest first.
payments	{ transactionReference: 1 }	Unique	Receipt retrieval and reconciliation.

Data integrity and validation

Defence in depth

Validation is applied at four levels, each catching what the level above cannot.

Level	Mechanism	What it protects against
1. Client	React form validation with inline messages	Usability only. Gives immediate feedback without a round trip. It is explicitly not a security control, because any client can be bypassed.
2. Route	express-validator chains running before the controller	Malformed, missing or out-of-range input. Produces the field-level errors the forms display.
3. Service	Business rule checks slot bookable, notice period, payment state	Logically valid requests that violate a domain rule, such as cancelling two hours before a visit.
4. Database	Mongoose schema types, enums, required fields, and unique indexes	Anything reaching MongoDB by any path, including a future administrative tool or a direct script. This is the only level that holds under concurrency.

Preventing double-booking

An application-level check read the slot, find it free, then write the booking leaves a window between the read and the write in which a second request can perform the same read. Under concurrent load this produces two confirmed appointments for one slot. The system therefore enforces uniqueness in the database itself:

```
appointmentSchema.index(
  { doctor: 1, appointmentDate: 1, appointmentTime: 1 },
  {
    unique: true,
    partialFilterExpression: {
      status: { $in: ["pending_payment", "confirmed", "completed"] }
    },
    name: "uniq_live_doctor_slot"
  }
);
```

The partial filter is what makes the constraint usable. A plain unique index would permanently burn the slot once an appointment was cancelled, because the cancelled record would still occupy the unique key. By restricting the constraint to the three statuses that actually hold a slot, a cancelled appointment is excluded from the index and its time becomes immediately bookable again. The service layer catches the resulting duplicate-key error and converts it into a 409 carrying a message the patient can act on: "This time slot was booked moments ago. Please choose another time."

Computed availability

Free slots are never stored. They are computed per request from three inputs: the doctor's recurring weeklySchedule, the appointments already held for that date and the current wall-clock time. A slot is returned as unavailable rendered as a disabled chip rather than hidden with the reason attached, so a patient can see that the time exists but is taken. This removes any possibility of stored availability drifting out of step with the booking records and eliminates the need for a nightly job to pre-generate slots.

System integration

Integration architecture

The React application communicates with the API exclusively through a service layer built on a single configured Axios instance. Two interceptors implement the client half of the contract.

- The request interceptor attaches the bearer token from storage, so no page component ever handles the token directly.
- The response interceptor unwraps the success envelope and converts any failure into a typed ApiError carrying a display-ready message and field-level errors. Components therefore never inspect err.response.data, and a 401 clears the stored token so the route guard redirects on the next render.

During development Vite proxies /api to the Express server on port 5000, which keeps the browser origin single and removes CORS from the development loop entirely. In production the same relative paths are served behind one origin.

Shared client state is held in three React contexts. AuthContext treats the token as the single source of truth and re-fetches the profile from /auth/me on start-up rather than trusting a cached user object, so a revoked session is detected before any protected screen renders. BookingContext holds the in-progress booking across the five screens of the journey, mirrored to sessionStorage so a mid-flow page refresh resumes where the patient left off.

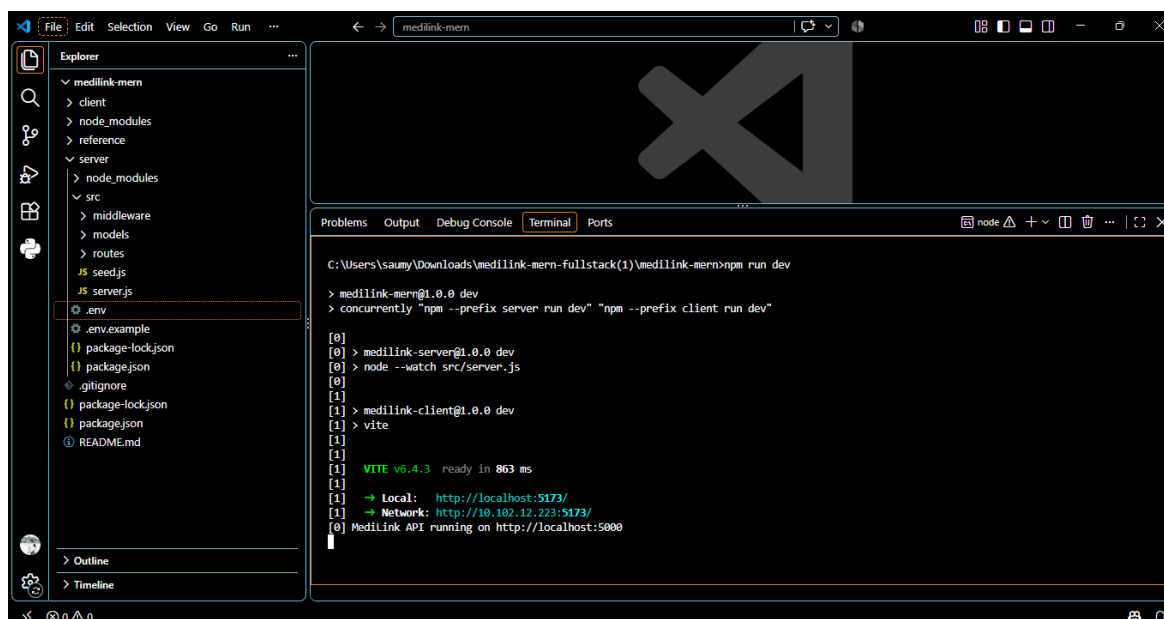


Figure 01: Backend and Frontend connected

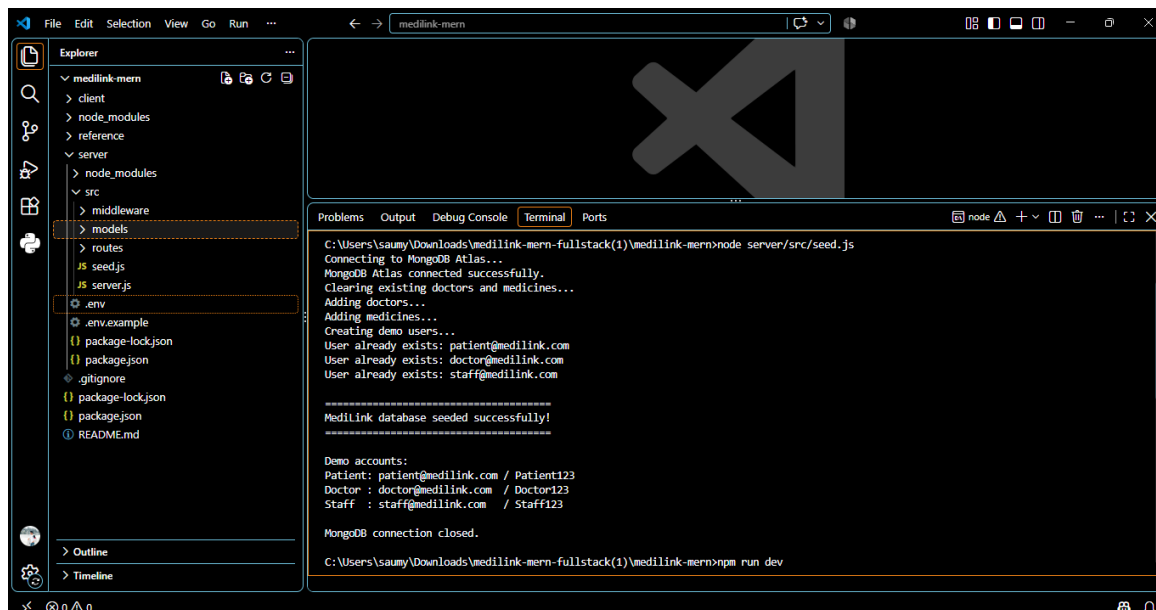


Figure 02: Database connected

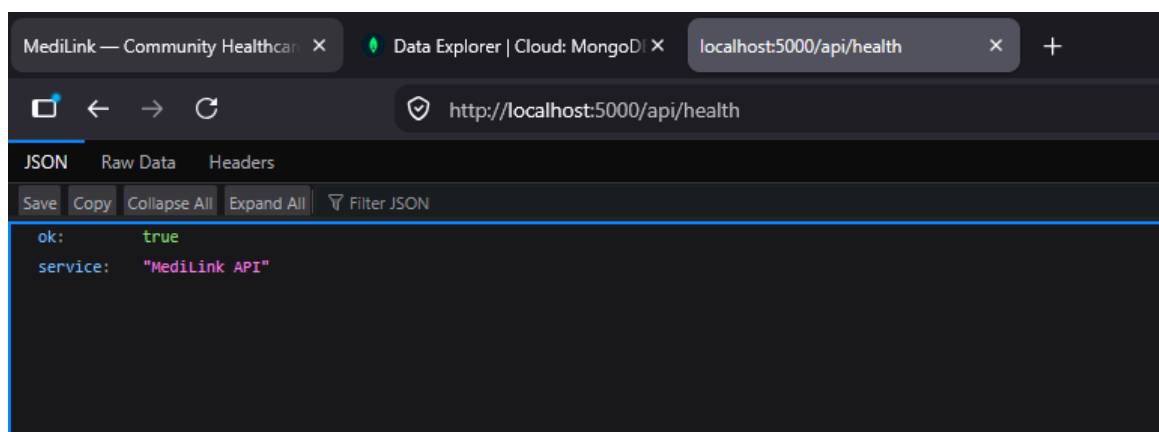


Figure 03: Backend API response

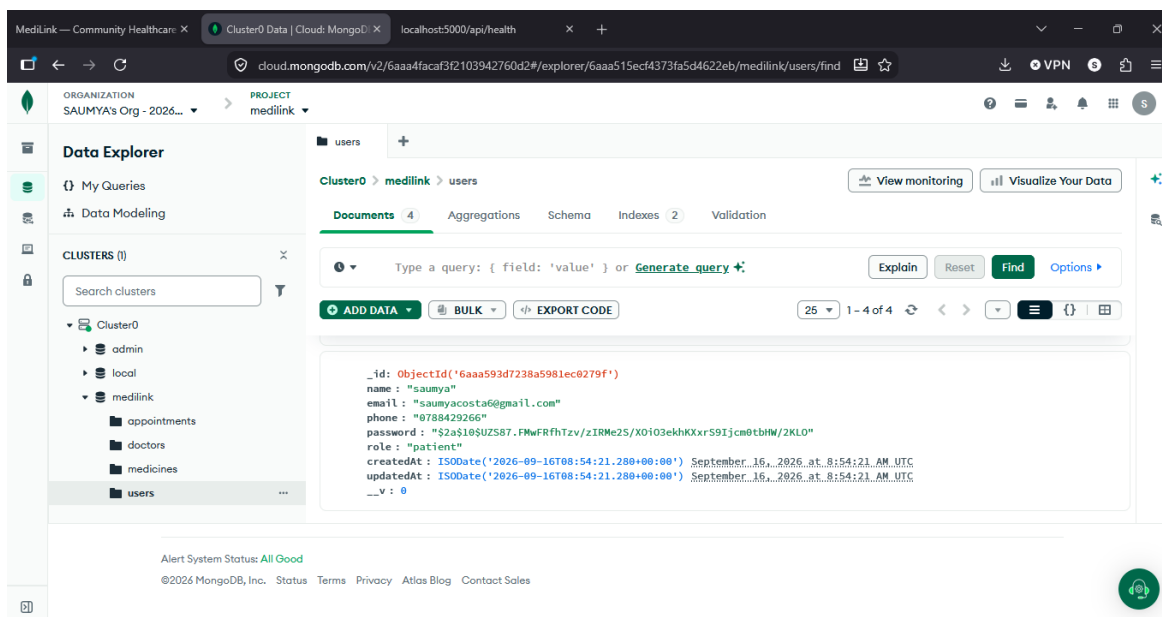


Figure 04: MongoDB Atlas database

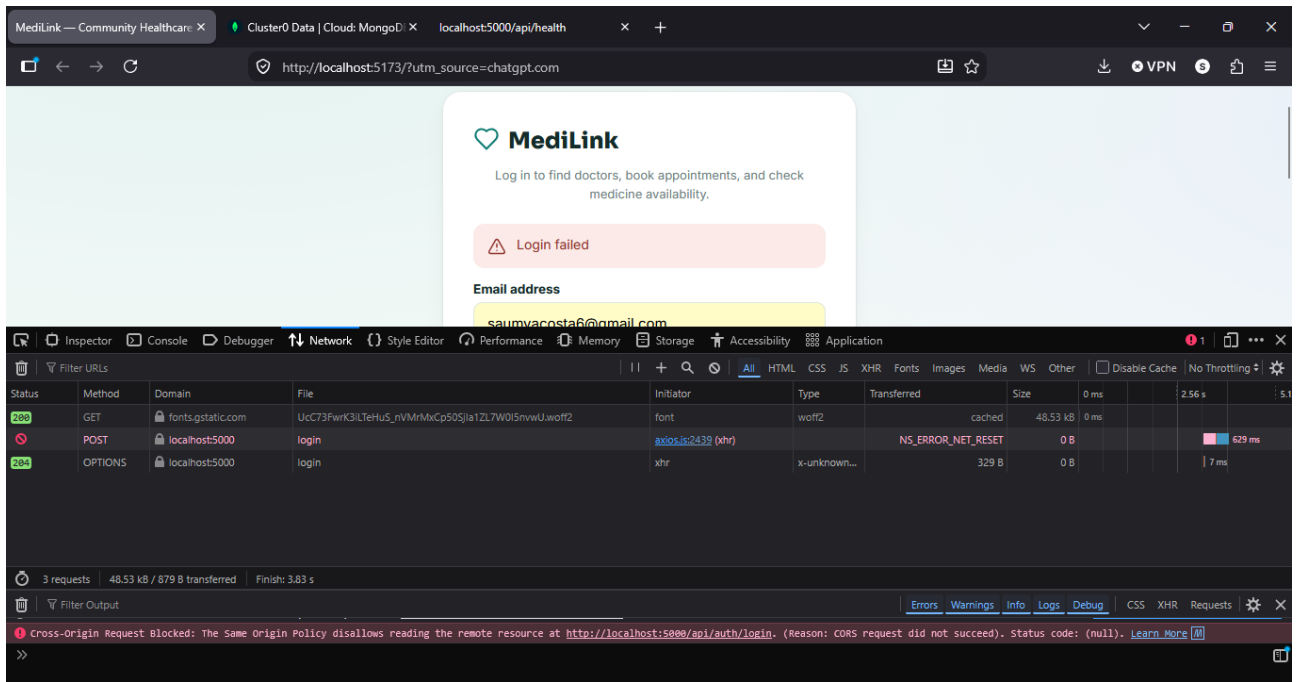


Figure 05: Frontend communicating with backend(login fail)

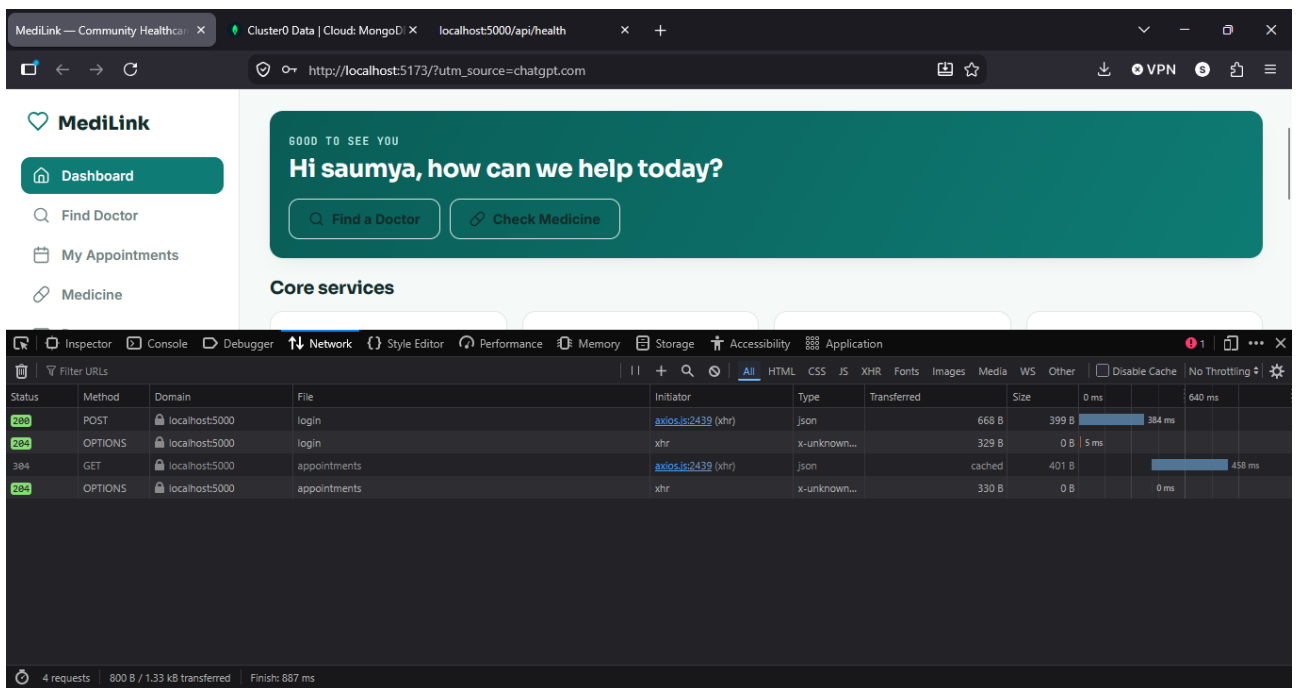


Figure 06: Frontend communicating with backend(login success)

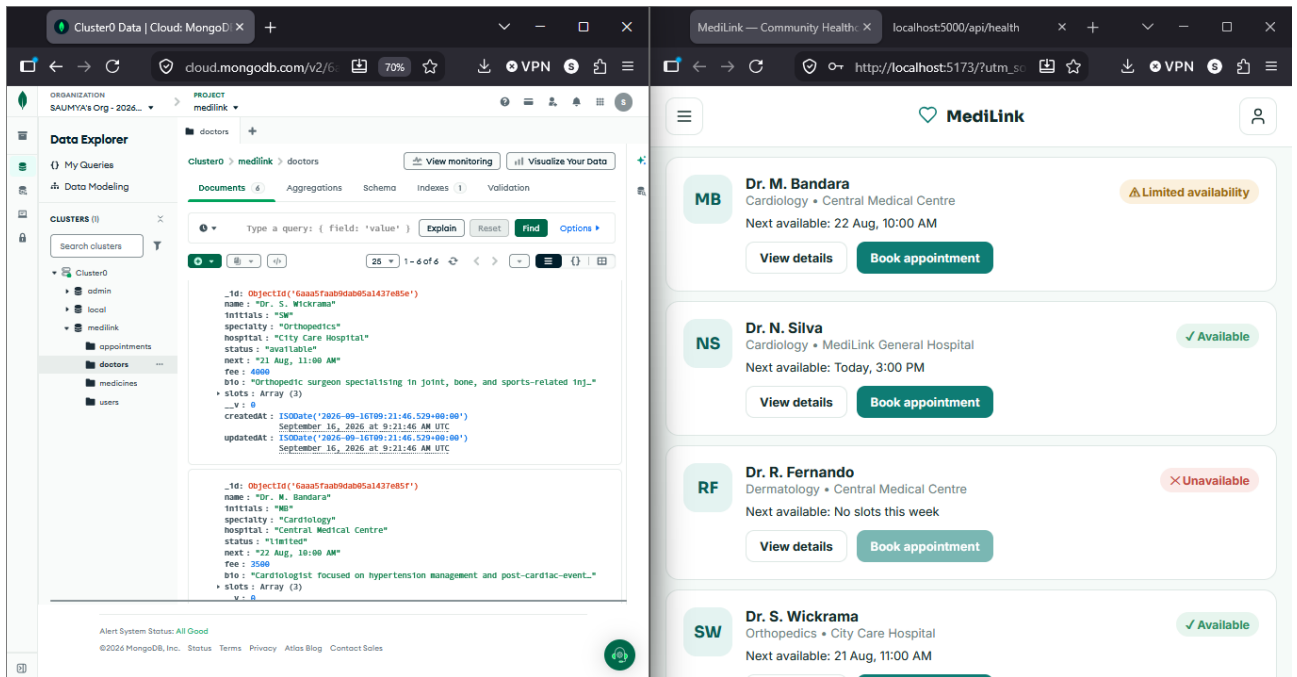


Figure 07: Doctors list view in Database & Interface

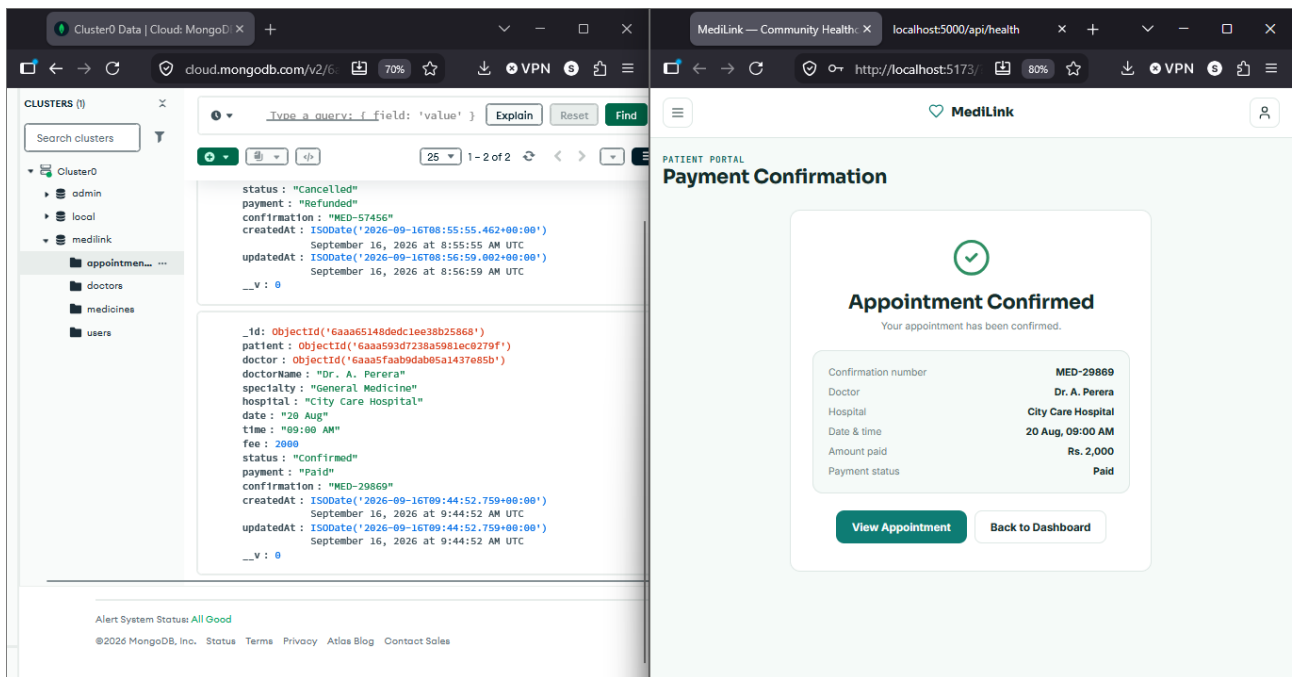


Figure 08: Payment details for appointment in Database & Interface

Completed back-end work

The table below summarises the server-side functionality implemented in this stage, grouped by feature area. All items listed are complete and exercised either by the automated suite or by manual testing through the interface.

Feature area	Completed back-end functionality
Application foundation	Layered Express application with configuration validated at start-up; MongoDB connection lifecycle with state reporting; structured logging that emits JSON in production; graceful shutdown on SIGINT and SIGTERM; health probe reporting uptime and database state.
Authentication and accounts	Registration with duplicate-email detection; bcrypt password hashing at cost factor 12; JWT issuing and verification with issuer validation; token-bearing middleware that re-loads the account so deactivated users lose access immediately; role-based authorisation; profile retrieval and update; password change requiring the current password.
Doctor discovery	Paginated search across doctor name, specialty, qualifications and hospital name; structured specialty and hospital filters; specialty aggregation with doctor counts; full doctor profile with fee and qualifications.
Availability engine	Slot generation from each doctor's recurring weekly schedule and configured slot duration; subtraction of appointments already held; suppression of past times; honouring of per-doctor blocked dates; next-available resolution scanning forward up to fourteen days; seven-day calendar with per-day free-slot counts.
Appointment booking	Slot reservation in pending_payment state with an immutable doctor and fee snapshot; database-enforced prevention of double-booking through a partial unique index; generation of patient-facing confirmation numbers; conversion of duplicate-key collisions into an actionable 409.
Appointment management	Upcoming and previous lists split server-side; single-appointment retrieval scoped to the owning patient; rescheduling validated against the same availability rules as a new booking, with an append-only history; cancellation with slot release, refund marking and a configurable minimum notice period.
Payment processing	Payment service mimicking an acquirer contract, with Luhn check-digit validation, expiry checking and deterministic decline rules; card brand detection; payments persisted as first-class records with gateway response codes; approval transitioning the appointment to confirmed; declines returned as 402 with the reservation retained; receipt generation; payment history; refund marking on cancellation.
Medicine availability	Quantity-based stock against a per-item reorder level; stock status derived by schema virtual so the badge cannot contradict inventory; search across brand and generic names; hospital filtering; relative last-updated timestamps; stock position aggregated per hospital.
Dashboard aggregation	Single-request dashboard payload built with a MongoDB aggregation pipeline, returning appointment counts by status, the next upcoming appointment and recent activity.
Validation and errors	Declarative validation chains on every mutating route; a central error handler normalizing ApiError, Mongoose validation, cast and duplicate-key failures into one envelope; correlation identifiers on every response; stack traces suppressed outside development.

Conclusion

The back-end implementation completes MediLink as a working platform. The interface designed by a Node.js and Express API over MongoDB and every screen that previously displayed fixed sample data now reads from and writes to persistent server-side state.

Twenty-one endpoints across seven resource groups implement patient accounts, doctor discovery, appointment availability, booking, rescheduling, cancellation, payment processing, medicine stock enquiry and dashboard aggregation. Six collections hold the data, with indexes declared against the queries the interface actually issues and integrity enforced at four levels from the browser to the database.

Three aspects of the work are worth resting because they go beyond wiring the screens to a data source.

1. Availability is computed rather than stored. Generating slots on demand from each doctor's consulting schedule, minus the appointments already held, removes an entire class of staleness bug and makes it structurally impossible for the interface to advertise a slot that is already taken.
2. Double-booking is prevented by the database, not by the application. A partial unique index restricted to the statuses that hold a slot enforces the constraint correctly under concurrent requests, while still releasing the time immediately when an appointment is cancelled.
3. Booking and payment are separate operations. This was a usability decision as much as a technical one: it means a declined card costs the patient a retry rather than their appointment slot.

The system also retains the interaction principles established during the design stage. Server error messages are written in the same plain language as the interface and are displayed verbatim, field-level validation errors are mapped onto the inputs that caused them, destructive actions remain behind a confirmation dialogue, and the persistent navigation means a patient is never trapped inside the booking sequence.

The payment gateway is simulated, token storage would need to change before deployment and notification delivery is outstanding. None of these affect the architecture: the payment service in particular was written against the contract a real acquirer presents, so substituting a live provider is a change confined to a single module.

MediLink now provides a complete and scalable foundation for a community healthcare platform, with the usability and interaction principles established during the HCI design process carried intact through to the working system.